

Proximal Policy Optimization Agent for 0DTE Options Execution

Architecture, MDP Formalization, Reward Engineering,
and Training Protocol for the RL Execution Layer

Kripta Studios Quantitative Research Division

March 2026

Contents

1	Introduction	2
2	Markov Decision Process Formalization	2
2.1	State Space	2
2.2	Action Space	2
2.3	Transition Dynamics	3
3	Actor-Critic Network Architecture	3
3.1	Network Design	3
3.2	Policy Distribution	3
4	Proximal Policy Optimization	4
4.1	Generalized Advantage Estimation	4
4.2	PPO Clipped Objective	4
4.3	Full PPO Objective	4
4.4	PPO Update Procedure	5
5	Reward Function Engineering	5
5.1	Profit and Loss Reward	5
5.2	Theta Decay Penalty	5
5.3	Stop-Loss Penalty	6
5.4	Move-Capture Bonus	6
5.5	Total Reward	6
6	Hard Exit Rules	6
7	Curriculum Learning Protocol	7

8	Training Infrastructure	7
8.1	Episode Collection and Rollout Buffer	7
8.2	Overfitting Detection	8
8.3	Model Persistence	8
8.4	Chunked Options Cache	8
9	Integration with GBT Classifier	8

1 Introduction

The GBT directional classifier (Stage 1) produces a probabilistic signal $\hat{y}_t \in \{\text{SHORT}, \text{HOLD}, \text{LONG}\}$ with associated confidence $c_t \in [0, 1]$. This signal answers the question: *which direction will the market move?* It does not, however, specify the optimal options contract to trade, the timing of entry within the observation window, or the exit strategy that maximizes risk-adjusted return.

These three sub-problems constitute the execution problem, which is solved by a Proximal Policy Optimization (PPO) [1] reinforcement learning agent. The RL agent operates on a real-options pricing environment (SPXOptionsEnv) populated with historical ThetaData quotes, and is trained to discover policies that select strikes, time entries, and manage exits so as to maximize a multi-component reward function that penalizes premature exits, excessive stop-losses, and theta-decay erosion.

The two-stage separation is a deliberate architectural choice: the GBT is exposed only to the directional classification objective, while the RL agent is exposed only to the execution objective given a fixed direction. This modularity allows each component to be retrained, replaced, or upgraded independently.

2 Markov Decision Process Formalization

2.1 State Space

Each episode begins when the GBT emits a directional signal with $c_t \geq c_{\min} = 0.55$. The RL agent’s state space $\mathcal{S}$ is the 182-dimensional vector defined in the Features document:

$$s_t = \left[x_t^\top \mid x_t^{\text{dyn}\top} \mid q_t^\top \mid c_t^\top \right]^\top \in \mathbb{R}^{182} \quad (1)$$

where $x_t \in \mathbb{R}^{163}$ is the shared market feature vector from the GBT, and the remaining 19 components encode dynamic market history, model context, and the position’s internal state. Prior to any position entry, position-specific components are set to zero.

Definition 1 (Episode Phases). *Each episode progresses through two phases:*

$$\underbrace{\text{ENTRY}}_{\text{strike selection}} \longrightarrow \underbrace{\text{IN_POSITION}}_{\substack{\text{hold/exit decisions} \\ \text{every 5 min}}} \longrightarrow \underbrace{\text{CLOSED}}_{\text{terminal}} \quad (2)$$

2.2 Action Space

The action space is discrete and phase-dependent:

ENTRY:

$$\mathcal{A}_{\text{entry}} = \{0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70\} \quad (3)$$

The entry action space consists of 7 delta buckets. For the selected delta bucket, the system chooses the actual strike from the real options chain that is closest to the bucket’s midpoint delta, given the GBT’s directional signal (call for LONG, put for SHORT).

IN_POSITION:

$$\mathcal{A}_{\text{pos}} = \{\text{HOLD}, \text{EXIT}\} \quad (4)$$

2.3 Transition Dynamics

The environment transitions follow the real options pricing data. At each timestep t while IN_POSITION, the mark-to-market value of the position is updated as:

$$P_t = \begin{cases} P_t^{\text{ThetaData}} & \text{if real quote available} \\ P_{t-1} + \Delta_{\text{pos}} \cdot (S_t - S_{t-1}) + \frac{1}{2}\Gamma_{\text{pos}} \cdot (S_t - S_{t-1})^2 & \text{otherwise (delta-gamma approx.)} \end{cases} \quad (5)$$

The unrealized P&L as a fraction of premium is:

$$P_L = \frac{P_t - P_{\text{entry}}}{P_{\text{entry}}} \quad (6)$$

3 Actor-Critic Network Architecture

3.1 Network Design

The PPOAgent is implemented as an actor-critic network with a shared feature extractor. The architecture contains approximately 181,010 trainable parameters:

Module	Input Dim	Output Dim	Parameters
Shared trunk (FC + LayerNorm)	182	256	46,848
Hidden layer 1 (FC + LayerNorm + Dropout)	256	256	66,048
Hidden layer 2 (FC + LayerNorm)	256	128	33,024
Actor head (entry, $ \mathcal{A}_{\text{entry}} = 7$)	128	7	903
Actor head (position, $ \mathcal{A}_{\text{pos}} = 2$)	128	2	258
Critic head (value)	128	1	129
Total			$\approx 181,010$

Table 1: PPO agent architecture. All FC layers use ReLU activation. Dropout $p = 0.1$ in hidden layer 1 during training.

The shared trunk processes the 182-dimensional state vector, producing a latent representation that is subsequently branched into the phase-conditioned actor heads and the value function critic. The actor selects from $\mathcal{A}_{\text{entry}}$ at entry and from $\mathcal{A}_{\text{pos}}$ during IN_POSITION; the critic outputs a scalar value estimate $V_{\phi}(s_t)$ used in the GAE advantage computation.

3.2 Policy Distribution

The actor produces a categorical distribution over the discrete action set:

$$\pi_{\theta}(a \mid s_t) = \text{Softmax}(f_{\text{actor}}(s_t)) [a] \quad (7)$$

During training, actions are sampled stochastically from π_θ . During evaluation and live trading, the policy is run deterministically ($a = \arg \max_a \pi_\theta(a \mid s_t)$) to reduce variance.

4 Proximal Policy Optimization

4.1 Generalized Advantage Estimation

The advantage function $\hat{A}_t$ measures how much better action a_t is relative to the average policy behavior from state s_t . GAE [2] computes a bias-variance tradeoff via exponential discounting of TD residuals:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1})(1 - \mathbf{1}_{\text{done}}) - V_\phi(s_t) \quad (8)$$

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} \quad (9)$$

where $\gamma = 0.99$ is the discount factor and $\lambda = 0.95$ is the GAE parameter, configured in `rl/config.py` under `RL_CONFIG`. The returns (rewards-to-go) are:

$$\hat{R}_t = \hat{A}_t + V_\phi(s_t) \quad (10)$$

4.2 PPO Clipped Objective

The probability ratio between the updated policy π_θ and the behavior policy $\pi_{\theta_{\text{old}}}$ used to collect the rollout is:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \quad (11)$$

The clipped surrogate objective prevents catastrophically large policy updates by constraining ρ_t to remain within $[1 - \epsilon, 1 + \epsilon]$, with $\epsilon = 0.2$:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t [\min(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (12)$$

4.3 Full PPO Objective

The total loss combines the policy objective, a mean-squared-error value function loss, and an entropy bonus to encourage exploration:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \cdot L^{\text{VF}}(\phi) - c_e \cdot H[\pi_\theta] \quad (13)$$

where:

$$L^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t [(V_\phi(s_t) - \hat{R}_t)^2] \quad (14)$$

$$H[\pi_\theta] = -\hat{\mathbb{E}}_t \left[\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t) \right] \quad (15)$$

with hyperparameters $c_v = 0.5$ (value coefficient) and $c_e = 0.01$ (entropy coefficient). The optimizer is Adam with learning rate $\text{lr} = 3 \times 10^{-4}$.

4.4 PPO Update Procedure

Algorithm 1 PPO Update Step

```

1: Collect  $N_{\text{ep}}$  episodes using current policy  $\pi_{\theta_{\text{old}}}$ 
2: Compute GAE advantages  $\{\hat{A}_t\}$  and returns  $\{\hat{R}_t\}$  (Eq. 9–10)
3: Normalize rewards via RunningMeanStd
4: for  $k = 1, \dots, K_{\text{epochs}}$  do
5:   for each mini-batch  $\mathcal{B}$  sampled uniformly from rollout buffer do
6:     Compute  $\rho_t(\theta)$  for all  $t \in \mathcal{B}$ 
7:     Compute  $L(\theta, \phi)$  via Eq. 13
8:     Update  $(\theta, \phi) \leftarrow (\theta, \phi) - \alpha \nabla L(\theta, \phi)$ 
9:   end for
10: end for
11:  $\theta_{\text{old}} \leftarrow \theta$ 

```

The training loop runs for `total_updates = 200` PPO update steps by default, with $N_{\text{ep}} = 64$ episodes per update and $K_{\text{epochs}} = 4$ optimization epochs over the collected rollout.

5 Reward Function Engineering

The reward function is a critical design choice for the RL execution agent. A naive reward based solely on terminal P&L would provide sparse feedback and encourage high-variance policies. The system employs a dense, multi-component reward R_t that decomposes the terminal objective into intermediate behavioral targets.

5.1 Profit and Loss Reward

The base reward at exit is proportional to the premium-normalized P&L:

$$R^{\text{PnL}} = P_L = \frac{P_{\text{exit}} - P_{\text{entry}}}{P_{\text{entry}}} \quad (16)$$

5.2 Theta Decay Penalty

Every timestep the position is held, a penalty proportional to the realized theta is applied:

$$R_t^\theta = -\alpha_\theta \cdot \frac{|\theta_{\text{pos}}| \cdot \Delta t}{P_{\text{entry}}}, \quad \alpha_\theta = 0.5 \quad (17)$$

This penalty discourages the agent from holding positions through large theta-decay periods, particularly in the final 90 minutes of the session when 0DTE theta is accelerating exponentially.

5.3 Stop-Loss Penalty

Hard stops imposed by the environment at $P_L < -0.50$ (50% premium loss) incur an additional penalty:

$$R^{\text{stop}} = \begin{cases} -1.0 & \text{if } P_L \leq -0.50 \text{ (hard stop triggered)} \\ 0 & \text{otherwise} \end{cases} \quad (18)$$

5.4 Move-Capture Bonus

The agent is rewarded for capturing a large fraction ρ of the maximum favorable move M_t available during the hold period:

$$M_t = \max_{\tau \in [t_{\text{entry}}, t_{\text{exit}}]} \frac{P_\tau - P_{\text{entry}}}{P_{\text{entry}}} \quad (19)$$

$$R^{\text{capture}} = \begin{cases} -0.2 \cdot (1 - \rho) & \text{if } M_t > 0 \wedge P_L > 0 \wedge \rho < 0.30 \\ 0 & \text{otherwise} \end{cases}, \quad \rho = \frac{P_L}{M_t} \quad (20)$$

A winning trade that exits with less than 30% of the available move captured receives a proportional penalty. This discourages premature profit-taking that leaves large residual gains unrealized.

5.5 Total Reward

The total reward signal combines all components:

$$R_t = \begin{cases} R^{\text{PnL}} + R^{\text{capture}} + R^{\text{stop}} & \text{if } t = t_{\text{exit}} \text{ (terminal)} \\ R_t^\theta & \text{otherwise (per-step)} \end{cases} \quad (21)$$

Remark 1. *The reward structure is designed to produce a patient agent that holds positions through minor drawdowns if the expected remaining upside is large, rather than exiting immediately at any positive P&L. The theta penalty counterbalances this patience near expiration, inducing natural time-based exit discipline.*

6 Hard Exit Rules

The environment enforces non-negotiable exit conditions that override the agent's action. These are implemented as pre-action checks in `SPXOptionsEnv.step()`:

Rule	Threshold	Rationale
Maximum loss	$P_L \leq -0.50$	Prevent ruin on a single trade
Maximum profit	$P_L \geq +4.00$	Capture extraordinary gains (400% of premium)
Time to close	$\text{minutes_to_close} \leq 5$	Avoid illiquid 15:55+ pricing
Maximum hold	180 minutes	Align with GBT lookahead horizon ($T_{\max}$)

Table 2: Hard exit rules enforced by `SPXOptionsEnv` regardless of agent action.

7 Curriculum Learning Protocol

The training curriculum follows a three-phase confidence progression managed by `CurriculumScheduler`:

Phase	Training %	Min Confidence $c_{\min}$	Rationale
Phase 0: High confidence	0–15%	0.60	Learn on cleanest signals first
Phase 1: Medium confidence	15–35%	0.55	Expand to moderate signals
Phase 2: Full distribution	35–100%	0.50	Learn full production range

Table 3: Curriculum learning schedule for RL training.

This protocol is motivated by the observation that high-confidence GBT signals produce more directionally-consistent price moves, providing cleaner reward signals for the PPO agent to learn from in the early stages of training. By the time the curriculum reaches the full confidence range, the agent has developed robust baseline policies that can be refined on the noisier low-confidence episodes.

8 Training Infrastructure

8.1 Episode Collection and Rollout Buffer

At each PPO update step, the trainer collects $N_{\text{ep}} = 64$ complete episodes using the current policy. Each episode is stored in a `RolloutBuffer` as a sequence of $(s_t, a_t, a_{\text{type}}, r_t, \log \pi_{\theta}(a_t|s_t), V_{\phi}(s_t), \text{done}_t)$ tuples. After collection, GAE returns are computed and rewards are normalized via a `RunningMeanStd` tracker.

State augmentation during training adds i.i.d. Gaussian noise $\epsilon_t \sim \mathcal{N}(0, 0.01)$ to continuous state components:

$$\mathbf{s}_t^{\text{aug}} = \mathbf{s}_t + \epsilon_t \quad (22)$$

This data augmentation improves robustness of the learned policy to the slight distributional shift between historical training data and live deployment conditions.

8.2 Overfitting Detection

Every `eval_interval = 20` update steps, the trainer runs a deterministic evaluation rollout of 50 episodes (no augmentation). Key metrics are compared between the stochastic training rollout (rolling average over the last 20 steps) and the deterministic evaluation:

$$\Delta_{\text{PF}} = \text{PF}_{\text{train}} - \text{PF}_{\text{eval}} \quad (23)$$

Warnings are emitted at:

- $\Delta_{\text{PF}} > 0.5$ and $\text{PF}_{\text{eval}} < 1.0$ severe overfitting.
- $\Delta_{\text{PF}} > 0.3$ mild overfitting, monitor.
- Eval win rate $< 40\%$ agent may be exhibiting degenerate policy.

The best model checkpoint (by evaluation profit factor) is saved to `rl_models/best_rl_agent.pt`.

8.3 Model Persistence

Checkpoints are saved at three levels:

1. `best_rl_agent.pt` best evaluation profit factor at any step.
2. `rl_agent_step_{N}.pt` periodic checkpoint every 50 steps.
3. `final_rl_agent.pt` end-of-training snapshot.

Training history is serialized to `rl_training_history.json`, containing per-step metrics for both training and evaluation rollouts.

8.4 Chunked Options Cache

The RL environment accesses historical options data via the `ChunkedOptionsCache`, which implements an LRU eviction policy parameterized by `max_days_in_ram` (default: 120 days, configurable via CLI). The cache key format is "YYYYMMDD_HH:MM", from which the date prefix is extracted to identify the correct daily shard. Memory consumption is bounded at $\mathcal{O}(k \cdot \bar{m}_d)$, where k is the cache size and $\bar{m}_d$ is the average shard size in memory.

9 Integration with GBT Classifier

The GBT ensemble and PPO agent are coupled through the `IntegratedTradingSystem` class in the live bot. The interface contract is:

1. GBT emits $(\hat{y}_t, c_t)$ every 5-minute bar.
2. If $\hat{y}_t \neq \text{HOLD}$ and $c_t \geq c_{\min}$, an episode is instantiated in the RL environment.
3. The RL agent steps through ENTRY and IN_POSITION phases using live options quotes from the real-time feed.

4. The episode terminates on agent EXIT action or hard exit rule trigger.

The `GBTEnsemble` exposes a PyTorch-compatible `__call__` interface that returns `(logits, dummy_time_pred)`, preserving compatibility with the RL state encoder that was originally designed for the legacy MLP:

$$\text{GBTEnsemble}(x) \rightarrow \left(\underbrace{\mathbf{p} \in \mathbb{R}^3}_{\text{class probs}}, \underbrace{(60.0, 0.0)}_{\text{dummy time pred}} \right) \quad (24)$$

The class probabilities $\mathbf{p}$ are passed as logits (no further softmax applied), since $\text{softmax}(\mathbf{p})$ where $\sum_k p_k = 1$ introduces only minimal distortion relative to $\mathbf{p}$ itself. This avoids requiring the RL state encoder to handle the log-probability transform that would be needed if true logits were passed.

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [2] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” *ICLR*, 2016.
- [3] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [4] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *NeurIPS*, vol. 30, 2017.
- [5] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Trans. Neural Networks*, vol. 12, no. 4, pp. 875–889, 2001.
- [6] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, “Deep direct reinforcement learning for financial signal representation and trading,” *IEEE Trans. Neural Networks and Learning Systems*, vol. 28, no. 3, 2017.
- [7] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed., Pearson, 2017.